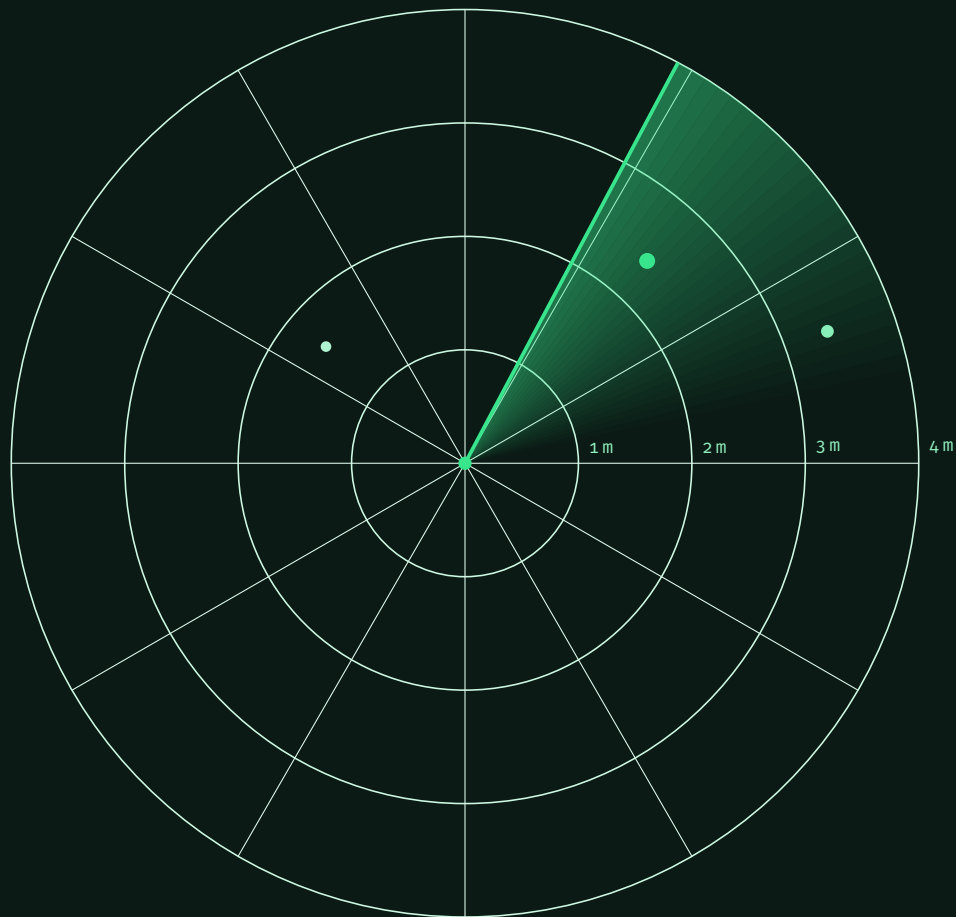




Radar Project Session 0

Session plans, build instructions and the function reference

Led by Harry Stevens · 2026/27

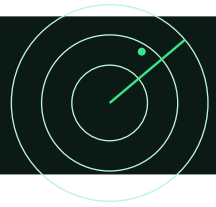
About this document. This document outlines and organises each session of the radar project. Each session has a short description, a list of activities and step-by-step instructions. The function reference at the back covers every function you will need for the Arduino radar.

Contents

Session 0: Mini Arduino Radar	2
Description	2
Activities	2
Instructions for activity: Mini Radar	2
Function Reference	8
Sketch structure	8
Digital pins	9
Analogue in, PWM out	10
Timing	10
Pulses and sound	11
Serial (USB) communication	11
Servo library	12
Maths helpers	13
Control flow	13
Display program (Python)	14

SESSION 0 // Monday 28/09/2026

Mini Arduino Radar



Description

We will begin this session with a brief introduction to the project. Harry will explain the absolute basics of how radar technology works. Once this short talk is over, we will immediately begin designing a small-scale, Arduino-based radar, which we will use to **detect an enemy vehicle and alert the defence personnel**. This should be completed during the time allocated.

This is a coding-heavy session, so extensive hardware knowledge is not required. If you are not confident with coding, Harry will be able to provide and teach you the code required to get the system working.

Activities



Instructions for activity: Mini Radar

1 Download the Arduino IDE

Search for **Arduino IDE** and open the result from docs.arduino.cc (circled in red below).

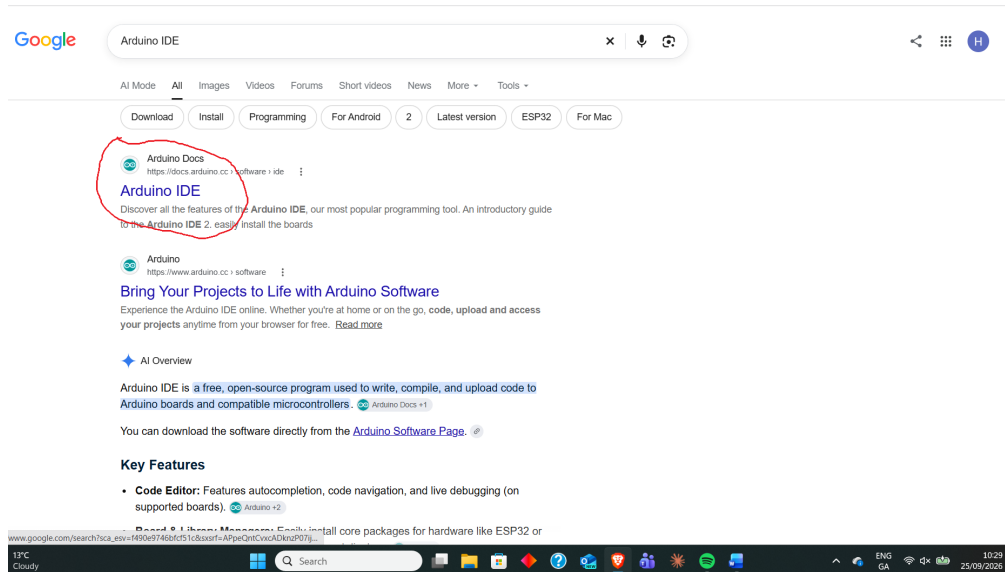
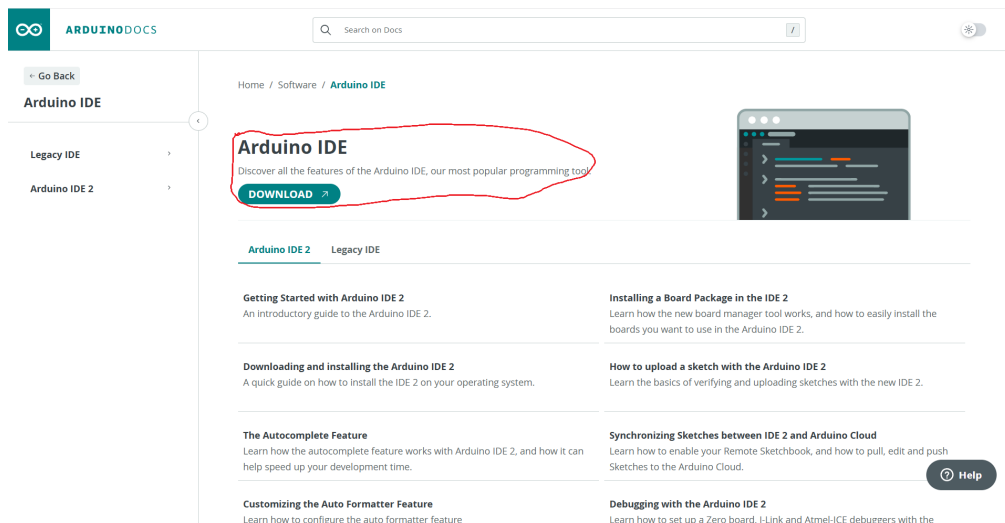


Figure 1: Search for “Arduino IDE” and open the Arduino Docs result.

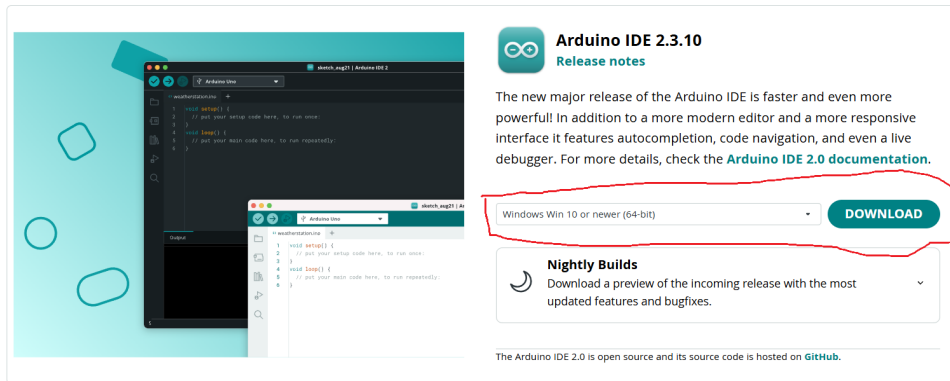
On the Arduino Docs page, click **Download**.



Click the Download button on the Arduino IDE page.

Pick the version that matches your computer. For most of us it will be the default option shown below (*Windows Win 10 or newer, 64-bit*).

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.10
Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit) **DOWNLOAD**

Nightly Builds
Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Choose your operating system, then click Download.

- Once it has downloaded, follow the installation instructions as normal.
- Click **Run** once the installation has finished.
- When it opens, the IDE may run a few extra installations. These can take a few minutes.

Before you upload Plug in your Arduino, then choose **Tools** → **Board** → **Arduino Uno** and **Tools** → **Port** (the COM port that appears when the board is plugged in). The ✓ **Verify** button checks your code. The → **Upload** button sends it to the board.

2 Begin coding

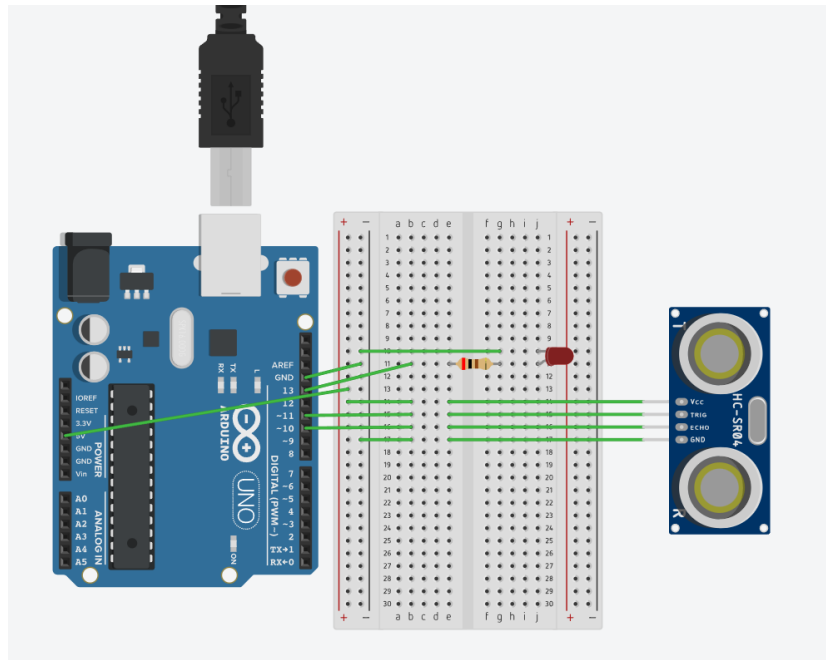
I understand that not everyone attending this project will know how to code. To get around this, I have broken the coding down into three levels: **beginner**, **intermediate** and **advanced**.

Beginner <i>never coded</i>	Intermediate <i>some coding</i>	Advanced <i>≈ COMP10010</i>
You will not be expected to code anything yourself. You will be given prewritten code with certain values missing. Your job is to fill in those values so the system behaves the way you want it to.	You will be given a program with only a few functions. Your job is to implement the missing functions, then iterate on your program to add functionality: extra LEDs, better resolution, or whatever you have in mind.	You will be given nothing. It is your responsibility to fully program the system. This level is for anyone comfortable with coding, roughly equivalent to having completed one coding module (COMP10010).

Just let Harry know which one you would like to try. **If you are unsure, start with beginner.** You can always try something harder once you finish it.

3 Begin testing

Once you are confident your program is ready to go, upload (flash) the code to your Arduino, then set up the breadboard as shown below.



Yours may look different depending on your pin selection.

HC-SR04 pin	Arduino pin	Purpose
VCC	5V	Powers the sensor
TRIG	D11	Arduino sends the 10 μ s “ping” pulse
ECHO	D10	Sensor replies with a pulse as long as the echo’s flight time
GND	GND	Common ground
LED	Digital pin + resistor	Warning light when a target is detected

Once that is done, play around with the radar and see what distance values you are getting. Then **measure that distance with a ruler** to see how accurate it is.

Test	Radar reading (cm)	Ruler (cm)	Error (cm)
1			
2			
3			
4			
5			

The sensor sends out a burst of 40 kHz sound and times how long the echo takes to come back. Sound travels at about 343 m/s, and it goes to the target *and back*, so

$$\text{distance (cm)} = \frac{\text{echo time } (\mu\text{s}) \times 0.0343}{2}.$$

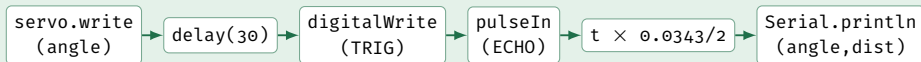
Our 24 GHz radar measures range the same way ($R = c\tau/2$), but with radio waves travelling at the speed of light instead of sound.

APPENDIX // ARDUINO C++ AND PYTHON

Function Reference

Every function you need to build the ultrasonic radar. Each entry gives the syntax, what the function does, a short example, where it fits in the radar, and the common mistake to avoid where there is one.

One radar measurement, in order



Session 0 uses steps 3–6 with the sensor fixed in place. Adding a servo (steps 1–2) turns it into a sweeping radar, and a Python program can then draw each angle, distance line on a radar screen.

Sketch structure

Arduino C++

void setup() { }

Runs **once** when the board powers up or resets. Configure pins, start Serial and attach the servo here.

```
void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT);
}
```

RADAR Pin setup, `Serial.begin`, `scanner.attach`.

void loop() { }

Runs **forever** after `setup()` finishes. When it reaches the end, it starts again from the top.

```
void loop() {
  // measure, decide, report... repeat
}
```

RADAR Holds the measurement (and the sweep loops once a servo is added).

#include <Library.h>

Pulls in a library's code. Goes at the very top of the sketch, outside any function.

```
#include <Servo.h>
```

RADAR Needed for the servo.

const int NAME = value;

A named constant. Use it for pin numbers so you change the wiring in one place, not all through the code.

```
const int TRIG_PIN = 11;
const int ECHO_PIN = 10;
```

RADAR TRIG, ECHO, LED and servo pin numbers.

int · long · unsigned long · float · bool

Variable types. On the Uno an `int` is 16-bit (−32,768 to 32,767). `unsigned long` is 32-bit and never negative. `float` holds decimals.

```
int angle = 90;
unsigned long duration;
float distanceCm;
```

RADAR Store `pulseIn` and `millis` results in `unsigned long`, and distance in `float`.

Watch out: An `int` overflows past 32,767. An echo time in μs can exceed that, so don't store it in an `int`.

type name(params) { return x; }

Your own function. It groups a job under one name so `loop()` stays readable. `void` means it returns nothing.

```
float measureDistanceCm() {
    // ...trigger and time...
    return duration * 0.0343 / 2;
}
```

RADAR `measureDistanceCm()` and `scanAt(angle)`.

Digital pins

D0–D13 · HIGH = 5V, LOW = 0V · avoid D0/D1 (USB serial)

pinMode(pin, mode)

Sets a pin as `OUTPUT` (drives it), `INPUT` (reads it) or `INPUT_PULLUP` (reads it with an internal resistor to 5V, so an unpressed button reads HIGH).

```
pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);
pinMode(2, INPUT_PULLUP); // button to GND
```

RADAR TRIG and the LED are outputs. ECHO is an input.

Watch out: Forgetting `pinMode` is the most common beginner bug.

digitalWrite(pin, HIGH | LOW)

Sets an output pin to 5V or 0V.

```
digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);
```

RADAR Makes the 10 μs trigger pulse and switches the warning LED.

digitalRead(pin) → HIGH | LOW

Reads whether an input pin is at 5V or 0V.

```
if (digitalRead(2) == LOW) {
    // button pressed (with INPUT_PULLUP)
}
```

RADAR An optional arm/disarm button for the radar.

Analogue in, PWM out

A0–A5 in · PWM (~) on 3, 5, 6, 9, 10, 11

analogRead(pin) → 0...1023

Reads a voltage from 0 to 5 V on A0–A5 as a 10-bit number, about 4.9 mV per step.

```
int raw = analogRead(A0);
float volts = raw * 5.0 / 1023.0;
```

RADAR A potentiometer that sets the alarm distance.

analogWrite(pin, 0...255)

Outputs PWM, a square wave whose duty cycle sets the average voltage. 0 is always off and 255 is always on. Runs at about 490 Hz (980 Hz on pins 5 and 6).

```
analogWrite(6, 128); // ~50% brightness
```

RADAR An LED that glows brighter as a target gets closer.

Watch out: Once a servo is attached, `analogWrite` no longer works on pins 9 and 10, because the Servo library takes over their timer.

Timing

16 MHz clock

delay(ms)

Pauses the whole program for a number of milliseconds. Nothing else runs while it waits.

```
delay(100); // ~10 readings per second
```

RADAR Time between pings, and servo settling time.

delayMicroseconds(us)

Pauses for a number of microseconds. Accurate up to about 16,000 µs.

```
delayMicroseconds(10);
```

RADAR Sets the width of the trigger pulse.

millis() → unsigned long

Milliseconds since power-up. Use it to time things without blocking. It rolls over after about 49.7 days.

```
unsigned long last = 0;
if (millis() - last >= 50) {
    last = millis();
    // take a reading
}
```

RADAR Blinks the alarm LED while still taking measurements.

micros() → unsigned long

Microseconds since power-up, with 4 µs resolution on the Uno. Rolls over after about 70 minutes.

```
unsigned long t0 = micros();
// ...
unsigned long dt = micros() - t0;
```

RADAR Measures how long one reading or sweep takes.

Pulses and sound

the time-of-flight core

pulseIn(pin, HIGH, timeout_us) → unsigned long

Waits for the pin to go HIGH, then returns how many microseconds it stayed HIGH. Returns 0 if nothing arrives before the timeout.

```
unsigned long d = pulseIn(ECHO_PIN, HIGH, 30000UL);
if (d == 0) { /* no echo */ }
float cm = d * 0.0343 / 2;
```

RADAR This is the range measurement. The ECHO pulse width equals the round-trip time of the sound.

Watch out: Always give a timeout. The default is 1 second, which freezes the program whenever nothing is in range. 30,000 µs covers about 5 m.

tone(pin, freq_hz [, duration_ms]) · **noTone(pin)**

Plays a square wave on a pin to drive a piezo buzzer. **noTone** stops it.

```
if (cm < 20) tone(8, 2000);
else noTone(8);
```

RADAR An audible alarm that alerts the defence personnel.

Watch out: **tone** stops PWM working on pins 3 and 11.

Serial (USB) communication

Arduino ↔ laptop / Pi

Serial is how the Arduino talks to the computer. The IDE's **Serial Monitor** shows the text and the **Serial Plotter** graphs numbers.

Serial.begin(baud)

Starts serial communication. The baud rate must match the Serial Monitor or Python script, or you get garbage.

```
Serial.begin(9600); // or 115200 for faster
```

RADAR Goes in **setup()**.

Serial.print(x [, decimals]) · Serial.println(x)

Sends a value as text. `println` adds a newline at the end. For floats, the second argument sets the number of decimal places (default 2).

```
Serial.print("Distance: ");
Serial.print(cm, 1);
Serial.println(" cm");           // Distance: 42.7 cm
```

RADAR Shows the measured distance, or one angle, distance line per reading for the display.

Serial.available() → int · Serial.read() → int

`available` is the number of bytes waiting from the computer. `read` takes one byte, or returns `-1` if nothing is waiting.

```
if (Serial.available() > 0) {
  char c = Serial.read();
  if (c == 's') armed = false;
}
```

RADAR Lets the laptop send commands such as “s” to stand down or “a” to arm.

Serial.parseInt() → long

Reads digits from the input and returns them as a number. Gives up after 1s if no number arrives.

```
int alarmCm = Serial.parseInt(); // type 30
```

RADAR Sets the alarm distance from the keyboard.

Servo library

```
#include <Servo.h> · brown = GND, red = 5V, orange = signal
```

Servo name; · name.attach(pin)

Creates a servo object (declare it outside any function), then connects it to a pin and starts sending control pulses.

```
Servo scanner;
void setup() { scanner.attach(9); }
```

RADAR The motor that points the sensor.

name.write(angle)

Moves the servo to an angle from 0 to 180°. The library converts this to a pulse 1–2 ms wide, sent every 20 ms.

```
for (int a = 15; a <= 165; a++) {
  scanner.write(a);
  delay(30); // let it arrive before pinging
}
```

RADAR Steps the beam across the field of view.

Watch out: Cheap servos buzz at 0° and 180°. Sweeping 15–165° avoids the end stops. On real hardware, power the servo from a separate 5V supply with a shared GND.

name.writeMicroseconds(us) · name.read() · name.detach()

`writeMicroseconds` sets the pulse width directly (roughly 1000–2000 µs) for finer control. `read` returns the *last angle you wrote*, not the measured position. `detach` stops the pulses so the servo goes limp.

```
scanner.writeMicroseconds(1500); // centre
```

RADAR Calibrating the servo so 90° really points straight ahead.

Maths helpers

built in

map(x, inLo, inHi, outLo, outHi)

Rescales a number from one range to another, using whole numbers only.

```
int bright = map(cm, 0, 100, 255, 0); // closer = brighter
```

RADAR Turns distance into LED brightness or buzzer pitch.

Watch out: Works in integers, so decimals are chopped off, and it does not clamp values that fall outside the range. Follow it with `constrain`.

constrain(x, lo, hi) · min(a, b) · max(a, b) · abs(x)

Clamp a value to a range, pick the smaller or larger of two values, and take the magnitude.

```
bright = constrain(bright, 0, 255);
closest = min(closest, cm);
if (abs(cm - lastCm) > 50) { /* glitch */ }
```

RADAR Keeps outputs in range and rejects sudden false readings.

sin(rad) · cos(rad) · radians(deg)

Trig functions that work in radians. `radians()` converts from degrees.

```
float x = cm * cos(radians(angle));
float y = cm * sin(radians(angle));
```

RADAR Converts polar (angle, range) to an x/y position.

Control flow

decisions and repetition

if (cond) { } else if (cond) { } else { }

Runs code only when a condition is true. Comparisons: `==` `!=` `<` `>` `<=` `>=`. Combine them with `&&` (and), `||` (or), `!` (not).

```
if (cm > 0 && cm < 20) {
    digitalWrite(LED_PIN, HIGH); // enemy detected!
} else {
    digitalWrite(LED_PIN, LOW);
}
```

RADAR Target detection and the alarm threshold.

Watch out: `=` assigns a value and `==` compares. `if (x = 5)` is always true.

```
for (init; cond; step) { } · while (cond) { }
```

for repeats a block a set number of times with a counter. **while** repeats for as long as a condition stays true.

```
for (int i = 0; i < 3; i++) { /* flash LED */ }
while (Serial.available() == 0) { /* wait */ }
```

RADAR Flashing alarm patterns, sweeps and waiting for commands.

```
type arr[N];
```

An array holds a list of values. Indexes start at 0.

```
float ranges[181];    // one per degree
ranges[angle] = cm;
```

RADAR Stores a full scan so you can compare sweeps and spot movement.

Display program (Python)

Python 3 on the Pi or a laptop · `pip install pyserial matplotlib`

The Arduino only sends numbers. The Python side reads them and draws the radar screen. It is also where the 24 GHz radar's signal processing (FFTs) will run later.

```
serial.Serial(port, baud, timeout=1)
```

Opens the Arduino's USB port. On Windows it looks like 'COM3' (check the IDE's port menu). On the Pi it is usually '/dev/ttyACM0'.

```
import serial
ser = serial.Serial('COM3', 9600, timeout=1)
```

RADAR Connects to the Arduino.

Watch out: Close the Arduino IDE's Serial Monitor first. Only one program can hold the port at a time.

```
ser.readline().decode().strip() · str.split(',') · float(x)
```

Reads one line of bytes up to the newline, converts it to text and removes the line ending. Then it splits the text at the comma and turns each part into a number.

```
line = ser.readline().decode(errors='ignore').strip()
try:
    angle, dist = map(float, line.split(','))
except ValueError:
    pass    # half a line at startup
```

RADAR Parses each angle,distance message.

```
plt.subplot(projection='polar') · plt.ion() · plt.pause(t)
```

A polar plot already looks like a radar screen. `ion()` turns on live mode, and `pause()` redraws the window each loop.

```
import math
import matplotlib.pyplot as plt
plt.ion()
ax = plt.subplot(projection='polar')
ax.set_thetamax(180); ax.set_rmax(200)    # cm
while True:
    # ...read angle, dist...
    ax.scatter(math.radians(angle), dist, s=8)
    plt.pause(0.001)
```

RADAR Draws the live sweep display.

Test your code first in the free Wokwi simulator (wokwi.com), then on real hardware.